Bilgisayar Porgramlama II

BLM2001-3

Örnek

- Komut dosyanız, kullanıcıdan [1 : 100] aralığında pozitif bir tamsayı değeri istemeye devam edecek ve bu girdinin asal olup olmadığını değerlendirecektir.

Hatırlatma – in işleci

- İn kelimesi İngilizce de içinde, dahil anlamında kullanılır
  - Phyton dili kullanımı da benzer
  - Bir stringin, dizinin, veri kümesinin içerisinde aranan bir değer için boolean geri dönüş verebilir
    - True - False

```
>>> sehir = "istanbul"
>>> 't' in sehir
True
>>> 'k' in sehir
False
>>>
```

```
yanit = input("Programdan çık? [E/H] ")
if ('E' in yanit) or ('e' in yanit) :
    print ("Çıkış başarılı..")
else:
    print ("Çıkış gerçekleşmedi!")
```

Liste Veri Türü

- Liste veri türünün klasik dizilerden farkı belirli bir türde veri içermek zorunda olmamasıdır.
  - Hem string hem sayısal veri içerebilir
  - Listenin üyeleri [ ] parantezler arasında , ile ayrılarak yazılır
  - String ifadeler " " işaretleri arasında belirtilmelidir
  - Sayısal veriler için ihtiyaç yoktur
    - Sayısal verilerde "" kullanılırsa ne olur?
- Liste = ["İstanbul","Kadıköy","Ata Caddesi",10,4,34722]
  - Len() işlevi listenin uzunluğunu verir

```
>>> Liste = ["İstanbul","Kadıköy","Ata Caddesi",10,4,34722]
>>> Liste
['İstanbul', 'Kadıköy', 'Ata Caddesi', 10, 4, 34722]
>>>
```

```
Liste = ["İstanbul","Kadıköy","Ata Caddesi",10,4,34722]
print("uzunluk=",len(Liste))
```

Liste Veri Türü – append() İşlevi

- Eklemek, iliştirmek anlamına gelen Append bir listenin sonuna eleman eklemek için kullanılır.

```
Liste = ["İstanbul","Kadıköy","Ata Caddesi",10,4,34722]
Liste.append("Türkiye")
```

- . İşlevi sonuna geldiği nesneye önüne geldiği işlevi uygular
  - Liste.append("Türkiye")
    - Liste nesnesine append işlevi uygulanıyor
- Append ile sadece bir eleman eklenebilir

```
Liste = ["İstanbul","Kadıköy","Ata Caddesi",10,4,34722]
Liste.append("Türkiye")
print("ilk eleman=",Liste[0])
print("Son eleman=",Liste[6])
print("Son eleman=",Liste[len(Liste)-1])
```

Lab Uygulaması

- Kullanıcıdan sırasıyla
  - Ad,
  - Soyad,
  - Cadde,
  - İlçe,
  - İl,
  - Ülke verilerini
- Alarak bir listeye ekleyen kodu yazınız

Liste Veri Türü – insert() İşlevi

- İnsert işlevi ile Listenin istenilen indisler arasına eleman eklenir
  - Liste[0] = "İş Adresi" ile karıştırılmamalı
  - Yerine geçmez, sonraki tüm elemanları kaydırır

```
Liste = ["İstanbul","Kadıköy","Ata Caddesi",10,4,34722]
Liste.insert(0,"İş Adresi")
Liste.insert(3,"Göztepe Mah.")
Print(Liste)
```

```
>>> print(Liste)
['İş Adresi', 'İstanbul', 'Kadıköy', 'Göztepe Mah.', 'Ata Caddesi', 10, 4, 34722]
```

Liste Veri Türü – extend() İşlevi

- Bir listeyi genişletmek amacıyla kullanılır
  - Bir listeyi başka liste ile genişletmek

```
Liste1 = ["1.Sınıf","2.Sınıf"]
Liste2 = ["3.Sınıf","4.Sınıf"]
Liste1.extend(Liste2)
print(Liste1)
```

Liste Veri Türü – remove() ve pop() İşlevleri

- Bir listeden elemanı kendisini belirterek çıkarmak için remove()

```
Liste1.remove("3.Sınıf")
print(Liste1)
```

  - Olmayan eleman çıkarılmak istenirse? Hata mesajı?

- Bir listeden elemanı indis numarası belirterek çıkarmak için pop()

```
Liste1.pop(0)
print(Liste1)
```

Liste Veri Türü – index() İşlevi

- Belirtilen elemanın listenin kaçıncı elemanı olduğu bilgisini verir

```
sehirler = ["Samsun","Amasya","Erzurum","Sivas"]
x=sehirler.index("Erzurum")
print("Sıra: ",x)
```

Liste Veri Türü – sort() ve reverse() İşlevi

- Sort listeyi sıralamak için kullanılır

```
sehirler.sort()
print(sehirler)
```

```
sayilar = [13,3,7,5,11,19,17]
sayilar.sort()
print(sayilar)
```

  - Sehirler ve sayilar listelerini birleştirip sıralayın

- Reverse işlevi listeyi ters çevirmek için kullanılır
  - Sıralama değil!!!!

```
sayilar = [13,3,7,5,11,19,17]
sayilar.reverse()
print(sayilar)
```

Liste Veri Türü – count() İşlevi

- Belirtilen elemanın liste içerisinde kaç kez geçtiği bilgisini verir

```
sayilar = [13,3,7,5,11,19,17,3]
s7= sayilar.count(7)
s3= sayilar.count(3)

print(s7,"ve",s3)
```

Uygulama

- Listelerde indis işlemleri
- Sehirler = ["Çanakkale","İzmir","Aydın","Muğla","Antalya","Mersin","Adana"]
  - Sehirler[-1] Adana
  - Sehirler[2:4] ['Aydın', 'Muğla']
  - Sehirler[:3] ['Çanakkale', 'İzmir', 'Aydın']
  - Sehirler[2:] ['Aydın', 'Muğla', 'Antalya', 'Mersin', 'Adana']
  - Sehirler[0:3] ['Çanakkale', 'İzmir', 'Aydın']
- Yukarıdaki satırları deneyiniz..

Random Modülü

- **import random**
- **randint(min,max)**
  - Min max aralığında integer olan bir sayı döner. Max dahildir. min <=n <=max
  - >>>random.randint(1,100)-86
- **randrange(min,max)**
  - Min ve max aralığında max dahil olmayan bir sayı döner. Min<=n<max
  - Bir parametre daha alır o parametre de bölünebilmeyi ifade eder.
    - >>> random.randrange(1,11,3) # 1 ve 11 arasında 3'e bölündüğünde 1 kalan bir sayı getir.

- **sample(liste,q)**
  - Liste içinde q adet rastgele değeri döner.
  - >>> sayilar = range(50)
    >>> random.sample(sayilar,3)
    [1, 19, 16]
    >>> random.sample(sayilar,3)
    [7, 10, 49]
    >>> random.sample(sayilar,3)
    [11, 37, 36]

- **choice(list)**
- Verdiğiniz bir liste içinden rastgele bir değer seçer.
- >>> liste = list(range(20)) # 0'dan 20'ye kadar sayılar gelsin.
  >>> liste # doğru gelmiş mi kontrol edelim.
  [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
  >>> random.choice(liste)
  0
  >>> random.choice(liste)
  5
  >>> random.choice(liste)
  10
  >>> random.choice(liste)
  11
  >>> random.choice(liste)
  5

Demet Veri Türü

- Listelere benzeyen bir veri türüdür (Tuple)
  - Temel Fark: Değişiklik yapılamaz
    - Eleman ekleme, çıkarma yapılamaz
  - calisan1 = "Ahmet","Yılmaz",1998,4500
  - calisan1 = ("Ahmet","Yılmaz",1998,4500)

```
>>> calisan1 = "Ahmet","Yılmaz",1998,4500
>>> type(calisan1)
<class 'tuple'>
>>> isim=calisan1[0]
>>> type(isim)
<class 'str'>
>>> maas = calisan1[3]
>>> type(maas)
<class 'int'>
```

Demet Veri Türü

- Demeti çözümleme

```
calisan2 = ("Ayşe","Güler",1997,5250)
isim,soyisim,dgmtarihi,maas=calisan2
```

  - Liste bu şekilde parçalara ayırılabilir mi?

- Hangi durumlarda demet kullanılmalı?
  - Demetler listelerin aksine değişiklik yapmaya müsait olmadıklarından listelere göre daha güvenlidirler.
  - Değiştirilmemesi gereken veriler içeren bir durumda liste yerine demetler kullanılmalıdır
  - Demetler listelere göre daha hızlı çalışır.

Sözlük Veri Türü

- Phyton dilindeki en kullanışlı veri türlerinden biri
  - Bir anahtar(key) yardımı ile verilere erişme mümkün
  - sozluk = {"key1":"veri1","key2":"veri2","key3":"veri3"}
  - **>>>** sozluk = {"elma": "meyve", "domates": "sebze", 1: "sayi"}
  - Anahtar üzerinden sözlükteki veriye erişilirken [] kullanılır.
  - “elma” bir “anahtar”, “meyve” ise bu anahtarın “değeri”dir. Aynı şekilde “sebze” değerinin anahtarı “domates”tir. Dolayısıyla Python’da sözlük; “anahtar” ve “değer” arasında bağ kuran bir veri tipidir.
  - Mesela bir adres veya telefon defteri yazmak istediğimizde bu sözlüklerden faydalanabiliriz